



Coding Challenges for MERN Interview

Table of Contents

- [JavaScript Array Problems](#)
 - [JavaScript String Problems](#)
 - [Algorithm Problems](#)
 - [Practical MERN Challenges](#)
 - [Solutions](#)
-

JavaScript Array Problems

Problem 1: Two Sum

Given an array of numbers and a target, return indices of two numbers that add up to target.

```
// Input: nums = [2, 7, 11, 15], target = 9
// Output: [0, 1] (because nums[0] + nums[1] = 2 + 7 = 9)
```

Problem 2: Remove Duplicates

Remove duplicates from an array and return unique values.

```
// Input: [1, 2, 2, 3, 4, 4, 5]
// Output: [1, 2, 3, 4, 5]
```

Problem 3: Flatten Nested Array

Flatten a deeply nested array.

```
// Input: [1, [2, [3, [4, 5]]]]
// Output: [1, 2, 3, 4, 5]
```

Problem 4: Array Intersection

Find common elements between two arrays.

```
// Input: [1, 2, 3, 4], [3, 4, 5, 6]
// Output: [3, 4]
```

Problem 5: Group By Property

Group array of objects by a property.

```
// Input:
const users = [
  { name: 'Alice', role: 'admin' },
```

```
{ name: 'Bob', role: 'user' },
{ name: 'Charlie', role: 'admin' }
];
// Output: { admin: [{...}, {...}], user: [{...}] }
```

JavaScript String Problems

Problem 6: Reverse String

Reverse a string without using built-in reverse.

```
// Input: "hello"
// Output: "olleh"
```

Problem 7: Palindrome Check

Check if a string is a palindrome (ignoring case and spaces).

```
// Input: "A man a plan a canal Panama"
// Output: true
```

Problem 8: Count Character Occurrences

Count occurrences of each character in a string.

```
// Input: "hello"
// Output: { h: 1, e: 1, l: 2, o: 1 }
```

Problem 9: Anagram Check

Check if two strings are anagrams.

```
// Input: "listen", "silent"
// Output: true
```

Problem 10: Longest Word

Find the longest word in a sentence.

```
// Input: "The quick brown fox jumps"
// Output: "quick" (or "brown" or "jumps" - all 5 chars)
```

Algorithm Problems

Problem 11: FizzBuzz

Print 1-100. For multiples of 3 print "Fizz", for 5 print "Buzz", for both print "FizzBuzz".

Problem 12: Fibonacci

Return nth Fibonacci number.

```
// fib(0) = 0, fib(1) = 1, fib(6) = 8
// Sequence: 0, 1, 1, 2, 3, 5, 8, 13...
```

Problem 13: Find Missing Number

Find the missing number in array of 1 to n.

```
// Input: [1, 2, 4, 5, 6] (n = 6)
// Output: 3
```

Problem 14: Valid Parentheses

Check if string has valid matching parentheses.

```
// Input: "{[()]}"
// Output: true

// Input: "{[{}]}"
// Output: false
```

Problem 15: Debounce Function

Implement a debounce function.

```
// debounce(fn, delay) - returns function that only executes
// after 'delay' ms have passed since last call
```

Practical MERN Challenges

Problem 16: Express CRUD API

Create a simple Express API for a Todo app with these routes:

- GET /todos - Get all todos
 - GET /todos/:id - Get single todo
 - POST /todos - Create todo
 - PUT /todos/:id - Update todo
 - DELETE /todos/:id - Delete todo
-

Problem 17: MongoDB Aggregation

Write an aggregation pipeline to:

- Group orders by customer
 - Calculate total amount per customer
 - Sort by total descending
 - Return top 5 customers
-

Problem 18: React Counter Component

Create a counter component with:

- Display current count
 - Increment, decrement, reset buttons
 - Don't allow count below 0
-

Problem 19: React Fetch Hook

Create a custom `useFetch` hook that:

- Fetches data from URL
 - Returns { data, loading, error }
 - Handles loading and error states
-

Problem 20: Full Stack User Auth

Design (pseudo-code) a user authentication system:

- Register endpoint
 - Login endpoint with JWT
 - Protected route middleware
 - React login form with context
-

Solutions

Solution 1: Two Sum

```
function twoSum(nums, target) {
  const map = new Map();
  for (let i = 0; i < nums.length; i++) {
    const complement = target - nums[i];
    if (map.has(complement)) {
      return [map.get(complement), i];
    }
    map.set(nums[i], i);
  }
  return [];
}
// Time: O(n), Space: O(n)
```

Solution 2: Remove Duplicates

```

// Method 1: Set
const removeDuplicates = arr => [...new Set(arr)];

// Method 2: Filter
const removeDuplicates = arr =>
  arr.filter((item, index) => arr.indexOf(item) === index);

// Method 3: Reduce
const removeDuplicates = arr =>
  arr.reduce((acc, item) =>
    acc.includes(item) ? acc : [...acc, item], []);

```

Solution 3: Flatten Array

```

// Method 1: flat()
const flatten = arr => arr.flat(Infinity);

// Method 2: Recursive
function flatten(arr) {
  return arr.reduce((acc, item) =>
    Array.isArray(item)
      ? acc.concat(flatten(item))
      : acc.concat(item), []);
}

// Method 3: Stack
function flatten(arr) {
  const stack = [...arr];
  const result = [];
  while (stack.length) {
    const item = stack.pop();
    if (Array.isArray(item)) {
      stack.push(...item);
    } else {
      result.unshift(item);
    }
  }
  return result;
}

```

Solution 4: Array Intersection

```

const intersection = (arr1, arr2) =>
  arr1.filter(item => arr2.includes(item));

// With Set (more efficient)
const intersection = (arr1, arr2) => {
  const set2 = new Set(arr2);

```

```
    return arr1.filter(item => set2.has(item));  
};
```

Solution 5: Group By

```
function groupBy(arr, key) {  
    return arr.reduce((acc, item) => {  
        const group = item[key];  
        if (!acc[group]) acc[group] = [];  
        acc[group].push(item);  
        return acc;  
    }, {});  
}  
  
// Usage  
groupBy(users, 'role');
```

Solution 6: Reverse String

```
// Method 1: Array  
const reverse = str => str.split('').reverse().join('');  
  
// Method 2: Loop  
function reverse(str) {  
    let result = '';  
    for (let i = str.length - 1; i >= 0; i--) {  
        result += str[i];  
    }  
    return result;  
}  
  
// Method 3: Reduce  
const reverse = str =>  
    str.split('').reduce((acc, char) => char + acc, '');
```

Solution 7: Palindrome

```
function isPalindrome(str) {  
    const cleaned = str.toLowerCase().replace(/[^a-z0-9]/g, '');  
    return cleaned === cleaned.split('').reverse().join('');  
}  
  
// Two pointer approach  
function isPalindrome(str) {  
    const cleaned = str.toLowerCase().replace(/[^a-z0-9]/g, '');  
    let left = 0, right = cleaned.length - 1;  
    while (left < right) {  
        if (cleaned[left] !== cleaned[right]) return false;  
    }
```

```

        left++;
        right--;
    }
    return true;
}

```

Solution 8: Count Characters

```

function countChars(str) {
    return str.split('').reduce((acc, char) => {
        acc[char] = (acc[char] || 0) + 1;
        return acc;
    }, {});
}

// Using Map
function countChars(str) {
    const map = new Map();
    for (const char of str) {
        map.set(char, (map.get(char) || 0) + 1);
    }
    return Object.fromEntries(map);
}

```

Solution 9: Anagram

```

function isAnagram(str1, str2) {
    const normalize = s => s.toLowerCase().split('').sort().join('');
    return normalize(str1) === normalize(str2);
}

// Character count approach
function isAnagram(str1, str2) {
    if (str1.length !== str2.length) return false;
    const count = {};
    for (const char of str1) count[char] = (count[char] || 0) + 1;
    for (const char of str2) {
        if (!count[char]) return false;
        count[char]--;
    }
    return true;
}

```

Solution 10: Longest Word

```

function longestWord(sentence) {
    return sentence.split(' ').reduce((longest, word) =>
        word.length > longest.length ? word : longest, '');
}

```

```

}

// Using sort
function longestWord(sentence) {
  return sentence.split(' ').sort((a, b) => b.length - a.length)[0];
}

```

Solution 11: FizzBuzz

```

function fizzBuzz(n) {
  for (let i = 1; i <= n; i++) {
    if (i % 15 === 0) console.log('FizzBuzz');
    else if (i % 3 === 0) console.log('Fizz');
    else if (i % 5 === 0) console.log('Buzz');
    else console.log(i);
  }
}

// One-liner array approach
const fizzBuzz = n => Array.from({ length: n }, (_, i) => {
  const num = i + 1;
  return (num % 3 ? ' ' : 'Fizz') + (num % 5 ? ' ' : 'Buzz') || num;
});

```

Solution 12: Fibonacci

```

// Recursive (slow)
function fib(n) {
  if (n <= 1) return n;
  return fib(n - 1) + fib(n - 2);
}

// Iterative (fast)
function fib(n) {
  if (n <= 1) return n;
  let prev = 0, curr = 1;
  for (let i = 2; i <= n; i++) {
    [prev, curr] = [curr, prev + curr];
  }
  return curr;
}

// Memoized
function fib(n, memo = {}) {
  if (n in memo) return memo[n];
  if (n <= 1) return n;
  memo[n] = fib(n - 1, memo) + fib(n - 2, memo);
  return memo[n];
}

```


Solution 13: Missing Number

```
function findMissing(arr) {
  const n = arr.length + 1;
  const expectedSum = (n * (n + 1)) / 2;
  const actualSum = arr.reduce((a, b) => a + b, 0);
  return expectedSum - actualSum;
}

// XOR approach
function findMissing(arr) {
  let xor = arr.length + 1;
  for (let i = 0; i < arr.length; i++) {
    xor ^= i + 1;
    xor ^= arr[i];
  }
  return xor;
}
```

Solution 14: Valid Parentheses

```
function isValid(s) {
  const stack = [];
  const pairs = { ')': '(', '}': '{', ']': '[' };

  for (const char of s) {
    if ('([{'.includes(char)) {
      stack.push(char);
    } else {
      if (stack.pop() !== pairs[char]) return false;
    }
  }
  return stack.length === 0;
}
```

Solution 15: Debounce

```
function debounce(fn, delay) {
  let timeoutId;
  return function(...args) {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => {
      fn.apply(this, args);
    }, delay);
  };
}

// Usage
const debouncedSearch = debounce((query) => {
```

```
    console.log('Searching:', query);
  }, 300);
```

Solution 16: Express CRUD

```
const express = require('express');
const app = express();
app.use(express.json());

let todos = [];
let id = 1;

app.get('/todos', (req, res) => res.json(todos));

app.get('/todos/:id', (req, res) => {
  const todo = todos.find(t => t.id === +req.params.id);
  if (!todo) return res.status(404).json({ error: 'Not found' });
  res.json(todo);
});

app.post('/todos', (req, res) => {
  const todo = { id: id++, ...req.body, completed: false };
  todos.push(todo);
  res.status(201).json(todo);
});

app.put('/todos/:id', (req, res) => {
  const index = todos.findIndex(t => t.id === +req.params.id);
  if (index === -1) return res.status(404).json({ error: 'Not found' });
  todos[index] = { ...todos[index], ...req.body };
  res.json(todos[index]);
});

app.delete('/todos/:id', (req, res) => {
  todos = todos.filter(t => t.id !== +req.params.id);
  res.status(204).send();
});
```

Solution 17: MongoDB Aggregation

```
db.orders.aggregate([
  {
    $group: {
      _id: '$customerId',
      totalAmount: { $sum: '$amount' },
      orderCount: { $sum: 1 }
    }
  },
  { $sort: { totalAmount: -1 } },
```

```

    { $limit: 5 },
    {
      $lookup: {
        from: 'customers',
        localField: '_id',
        foreignField: '_id',
        as: 'customer'
      }
    },
    { $unwind: '$customer' },
    {
      $project: {
        customerName: '$customer.name',
        totalAmount: 1,
        orderCount: 1
      }
    }
  }
]);

```

Solution 18: React Counter

```

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(c => c + 1)}>+</button>
      <button onClick={() => setCount(c => Math.max(0, c - 1))}>-</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}

```

Solution 19: useFetch Hook

```

function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const controller = new AbortController();

    const fetchData = async () => {
      try {
        setLoading(true);
        const res = await fetch(url, { signal: controller.signal });
        if (!res.ok) throw new Error('Failed to fetch');
      }
    };
  });
}

```

```
        const json = await res.json();
        setData(json);
    } catch (err) {
        if (err.name !== 'AbortError') setError(err.message);
    } finally {
        setLoading(false);
    }
};

fetchData();
return () => controller.abort();
}, [url]);

return { data, loading, error };
}
```

Practice these problems to build coding confidence for your interview!